Task Heads Chosen by a Language Model from What They Decode

Filippo Merlo

Draft — November 7, 2026

Abstract

In the companion document (`proposal-simple`) lexical task heads are chosen by fixed rules: a hand-written term list (Yang et al., 2026), word lists built from the words that all heads decode, or a causal score (Davidson et al., 2025). Here a reasoning language model reads, for every attention head of Llama-3.1-8B-Instruct, the tokens the head decodes on a task's correctly answered prompts, and scores each head for how clearly it represents the task. The heads that write the task's answer are then removed, and the highest-scored heads form the sets. Heads are scored from J-head and from Logit-head readings, with all heads in one request or in batches. The sets are tested with the two causal checks of the companion document, on items not used to choose them, beside the head sets chosen there by fixed rules. A pilot on one task comes first.

1 Idea and questions

Every rule used so far to choose heads looks for given words in what a head decodes. Yang et al.'s term lists are written by hand (`proposal-simple`, Sec. 3, Definition 1). The decoded-word lists of `proposal-simple`, Secs. 15–16 are counted from all heads' readings, but they are still fixed lists of ten words matched token by token. A list misses heads that decode the task in other words, and it cannot tell a head that decodes "capital" on every prompt from one that decodes it next to the capital itself. A person reading a head's top tokens makes both judgements easily. Here a reasoning LLM makes them, for every head: it is given each head's most frequent decoded tokens on the task's prompts and scores how clearly they represent the task. Whether a head writes the answer is decided afterwards, from the readings, and such heads are removed before the sets are formed.

Q1 Do the highest-scored heads carry the task in the two causal checks, on items not used to choose them, and how do they compare with the sets of `proposal-simple`, Sec. 16 of the same size (E1, CC, FV, random)?

Q2 Are the heads scored from J-head readings better or worse than those scored from Logit-head readings, and how much do the two sets overlap?

Q3 Does the LLM score better when it sees fewer heads at a time (batches of 128) than when it sees all 1,024 at once?

Q4 Does removing the heads that write the answer change the effect of the sets?

No hypotheses are tested; accuracies and set measures are reported, as for E4 (`proposal-simple`, Sec. 16.2).

2 What is carried over

Model, tasks, prompts and scoring are those of `proposal-simple`: Llama-3.1-8B-Instruct, the 14 tasks and Yang et al.'s five instruction templates (`proposal-simple`, Sec. 4.2), the two readouts (`proposal-simple`, Sec. 3), the flexible score (`proposal-simple`, Sec. 4.5). The design of E4 (`proposal-simple`, Sec. 16) is used unchanged wherever it applies: the item split into halves A and B (seed 0), two folds (choose on A, test on B, and the reverse), the identification prompts P^+ of each half (the first 100 prompts of each rule template answered correctly), the two causal checks with the rule-style means of the choosing half and the control mean in both checks, the ten values of α, replies of at most 16 tokens, and the cross-fitted choice of α. The LLM sees readings of the choosing half only.

3 Readings

For each task, half and rule template, the prompts of P^+ are read again as in `scripts/heldout_readings.py`, with Yang et al.'s batches of 20 and the padding masked, but each head's top 20 tokens are kept instead of its top 10, under both readouts, at the last prompt token. The top 10 of these readings must equal the stored readings of E4; the agreement is reported. The five templates' prompts are pooled: up to 500 prompts per task and half.

4 The head table

For each task, half and readout, every head (l, h) gets one row with three entries, computed over the pooled prompts P:

- *answer share*: the share of prompts $p \in P$ whose own answer's first token is among the head's 20 tokens on p. This is the retrieval-head test of Yang et al. (2026) (`proposal-simple`, Sec. 11) at $k = 20$; the answer is tokenized with its leading space, as in their code.
- *input share*: the same with the first token of the item's input, also with a leading space, as the input stands in the templates.
- *tokens*: the 20 tokens most often among the head's 20 over P, each with its share of prompts (ties by token id).

Tokens are written as JSON strings, so that a leading space and special characters are kept. Shares have two decimals. Each head is given an anonymous id (`H0000` to `H1023`) by a random permutation (seed 0, the same for both readouts and halves), and rows are listed in the order of the ids, so that neither the id nor the position tells the LLM where a head sits. Example rows are in App. B.

5 Scoring heads with an LLM

The LLM is told what the task is (a one-sentence description, the five templates, and ten input–answer pairs drawn with seed 0 from the choosing half's correct prompts) and what was measured, and is asked to give every head of its table a score from 0 to 100 for how clearly the head's tokens, across the prompts, represent the task (App. A). A head scores high when its tokens name the task or the relation between input and answer, or the kind of answer without being answers ("city" for capitals, "German" for a translation into German); words that name the task count even when they are in the instruction, as in Yang et al.'s term lists. A head scores 0 when none of its tokens has to do with the task, and low when its tokens are only answers

or only inputs, which by themselves do not represent the task. The LLM is told not to lower a head's score because it also shows answers or inputs, and to mark such heads instead in two lists (*answer*, *input*), decided by reading the tokens, not from the shares. For every head scored 50 or more it names the tokens that support the score and gives one sentence. A worked example uses a task outside the study (chemical symbols). The LLM is not told which readout the table comes from: both readouts get the same prompt.

Two conditions.

- **All**: one request with all 1,024 rows, which scores every head.
- **Batched**: the 1,024 ids are cut, in id order, into 8 batches of 128, so that each batch mixes heads from all layers; one request per batch scores its 128 heads, with the same prompt. The 8 batches' scores are put together. Scores given in different requests may not be on the same scale; the comparison with the *all* condition shows whether this matters.

From scores to head sets. After the requests, by script: every head with an answer share of 0.10 or more is removed, the ρ of Definition 1 (`proposal-simple`, Sec. 3), so that a removed head is one that would be a retrieval head of Yang et al. (2026) at $k = 20$. The remaining heads are ordered by score, ties by the order of their anonymous ids (which is random), and the first 20, 30 and 40 form the sets; the three sets of a condition are therefore nested. The input share does not remove heads; it is shown to the LLM and reported. The LLM's own *answer* flag is not used to remove heads either: it is recorded and compared with the answer share. Because the removal comes after the requests, other removal rules can be tried on the same scores without asking the LLM again.

Model and settings. GPT-5.5 (Azure OpenAI deployment `gpt-5.5`, model `gpt-5.5-2026-04-24`), reasoning effort *medium*, at most about 128,000 completion tokens (the reasoning counts against them). A reply is checked by script: valid JSON, exactly one entry for each head of the table, scores whole numbers from 0 to 100, both flags true or false, tokens and a sentence for every head scored 50 or more. A reply that fails is asked again at most twice, with the error appended to the prompt. If Azure's Responses API accepts a reasoning summary for this deployment, it is used, so that the summary is kept too; otherwise chat completions. The reasoning itself is not returned by the API, only its token count and, with the Responses API, a summary.

Stability. In the pilot, every request of the *all* condition is made a second time with the same prompt, to measure how much the scores change from one request to the next: the rank correlation (Spearman) of the two sets of scores, and the Jaccard overlap of the sets of 20, 30 and 40 heads. Only the first request's sets are patched.

What is recorded. Every output of the LLM is kept, including the replies that fail the check:

- `data/llm_heads/calls.jsonl.gz`, one line per request: task, fold, readout, condition, batch, attempt, time, request settings, the prompt's file and its SHA-256, the whole response body (text, reasoning summary if any, token usage with the reasoning tokens, model version, response id), and the result of the check.
- `data/llm_heads/prompts/`: the exact prompt of every request; `idmap/`: the anonymous ids; `batches/`: the batches.
- `data/llm_heads/scores/`: every head's score and flags, with its layer, head, answer share and input share.

- `data/review/llm_heads_{task}.md`: for reading by hand, for each readout, fold and condition, the heads by score with the tokens shown, the reasons, the flags and both shares, the heads removed as answer heads, and the sets.

A request is reused, not repeated, when a logged request has the same prompt.

6 Head sets

For each task and fold:

- **LLM sets**: All-J-N, All-L-N, Bat-J-N, Bat-L-N for $N \in \{20, 30, 40\}$ (condition, readout, size), after the answer heads are removed.
- **Larger LLM sets**, added in the pilot after the first patching: the first 60 and 80 heads and as many heads as the fold's full CC set (N_t), only where every head of the set has a score above 0 in both folds (beyond that the order is the random order of the heads scored 0), with CC-60, CC-80, FV-60, FV-80 and E4's CC and FV-N_t as references.
- **LLM sets without removal**, in the pilot: the same from the scores alone, to see what the removal changes (Q4).
- **Random sets**: for each LLM set after removal, five draws with the same number of heads in each layer, drawn outside it (seeds 0–4), as in E4.
- **References** from E4 on the same items: E1, CC-20, CC-40, FV-20, FV-40 and their random draws, already run (`proposal-simple`, Sec. 16); CC-30 and FV-30, the first 30 heads of the same rankings, added here so that every size has references.

7 Causal checks

Both checks of E4 on the testing half, unchanged (`proposal-simple`, Sec. 16.1): check 1 patches the zero-shot prompts of the first 100 items answered correctly with one shot; check 2 the 1-shot prompts of the first 100 answered wrongly; the five rule-style means of the choosing half and the control mean in both checks; $\alpha \in \{-10, -4, 1, 2, 3, 4, 5, 6, 8, 10\}$; replies of at most 16 tokens, scored by the flexible score, with the first-token score beside it. Random sets: check 1, task means only. Every patched reply is saved (`answers/`) and checked for answers that are the same for every item before any accuracy is read.

8 What is reported

Accuracies only, as for E4: per task, check and set, the accuracy at the cross-fitted α averaged over the two folds, the control mean at the same α, the unpatched accuracy, and accuracy against α in figures; the LLM sets beside the references of the same size (Q1–Q3), and with and without the removal (Q4). Set measures:

- overlap (Jaccard) between All-J and All-L, Bat-J and Bat-L, All and Bat, at each size; overlap with E1, CC-N and FV-N; rank correlation of the scores between readouts and between conditions;
- how many of the highest-scored heads are removed as answer heads; agreement of the LLM's *answer* and *input* flags with the answer and input shares;
- the distribution of the scores, and the scores of heads with a high answer or input share;

- copy share (`proposal-simple`, Sec. 4.6) of the tokens named in support of a score;
- layers of the heads in each set;
- tokens, requests, retries and cost per condition.

9 Pilot

The method is first run on one task, English–German, with both readouts, both folds and both conditions. It is the task on which no set of 20 or 40 heads of E4 has an effect in the first check, while the task can be carried by heads (Table 1). If an LLM set of this size has an effect there, it comes from which heads are chosen. On the held-out items of E4, the first check's unpatched accuracy is 0.00, E1, CC-20, CC-40, FV-20, FV-40 and their random draws reach at most 0.02, and the heads ranked by their causal effect reach 0.01 even at the size of the CC set; only the full CC set, with 97 to 99 heads, has an effect (0.39). Heads that decode words of the task exist: the task words of E4's word list for this task are *german*, *germany*, *germans*, *english*, *berlin*, *translation*. The task has 100 patched items per fold in both checks, and its control task is English–French. Its choosing half gives 500 identification prompts (100 per template).

	E1	CC-20	CC-40	FV-20	FV-40	CC (all)	FV, CC's size
Check 1	0.02	0.00	0.01	0.00	0.00	0.39	0.01
Check 2	0.20	0.13	0.18	0.16	0.24	0.21	0.17

Table 1: English–German in E4 (`proposal-simple`, Sec. 16.2): accuracy at the cross-fitted α, mean of the two folds. Unpatched accuracy 0.00 in both checks; random sets of 20 and 40 heads 0.00 in check 1 and 0.07–0.08 in check 2.

What the pilot gives up is a known task head to check the scores against, as L16H25 is for country–capital (`proposal-simple`, Sec. 15). A rise on English–German is informative by itself; no rise could mean that the method fails or that the task is hard to carry with 20 to 40 heads. In that case country–capital is run next, where L16H25 must score high under both readouts: in the stored top-10 readings of half A it decodes " cities", " city", " towns" and " capital" on all or most prompts, with answer and input shares of 0.

The pilot stops twice for a reading together: once the tables are built (one table excerpt, the exact token counts and cost, before any request), and once the LLM has scored (the review file, before patching).

What the pilot is to show. Stated before it is run, and reported one by one, not as a pass or fail on accuracy:

1. the replies pass the check, with at most one retry per request;
2. the scores agree with the tokens shown and the reasons with the scores, on a reading by hand of the review file; the LLM's *answer* flags agree with the answer shares;
3. how many of the highest-scored heads are answer heads, and what removing them changes in the checks;
4. the accuracies of the LLM sets in both checks, beside the references of the same size, with the patched answers checked for constant answers;
5. the stability of the scores across two requests;
6. the cost of each condition, measured.

Then it is decided whether to change the prompt, drop a condition, or run all 14 tasks.

10 Cost

Prices: $5 per million input tokens, $30 per million output tokens; reasoning tokens are billed as output. Input, measured on the English–German prompts with the tokenizer of GPT-4o and GPT-5 (`o200k_base`; that of GPT-5.5 is not public): 140,000 to 144,000 tokens for a request over all heads, 18,200 to 19,000 for a batch of 128; 1.73 million for the pilot. The scores take about 25 tokens per head (25,000 for all heads, 3,000 per batch). The number of reasoning tokens is the most uncertain part: the ranges below assume 20,000 to 60,000 for a request over all heads and 5,000 to 20,000 for a batch, estimated for high effort, and are an upper bound at medium effort. The full run assumes tables of the same size for the other tasks. One selection is one task, readout and fold.

	requests	input	output	cost
All, per selection	1	142k	45–85k	$2.1–3.3
Batched, per selection	8	8 × 19k	8 × 8–23k	$2.6–6.3
Pilot (4 selections per condition, with the repeat)	40	1.73M		$27–51
14 tasks (56 selections per condition)	504	16.3M		$265–535
Repeat in the full run, if wanted	52	7.4M		$105–170

Table 2: Cost of the LLM requests: input measured, output estimated (an upper bound at medium effort). The pilot measures it.

If the full run is too costly, three cuts: reasoning effort *low* in the batch requests; leaving out of each row the tokens seen on fewer than 1% of the prompts (mostly single occurrences), which should shorten the tables by about a third to a half; the batched condition only on the tasks where the *all* condition does poorly.

GPU. Pilot: under 10 minutes of readings, one to two hours of patching on one GPU (24 LLM sets, 2 references and 60 random sets per fold). All 14 tasks: about 30 minutes of readings and 6 to 8 hours of patching on four GPUs, most of it the random sets.

11 Plan

1. This draft and the prompt (App. A), read together and settled.
2. Pilot on English–German: readings, tables (stop), requests (stop), patching, report.
3. Decision on the prompt and the conditions; then all 14 tasks, the results here, and a row in `summary-simple`.

A Prompt

One prompt for both conditions. Placeholders between « and » are filled by script: `DESCRIPTION` with the task's description (`materials/generation/e4b_tasks.json`, the one written for E4b), `TEMPLATES` with the five templates as JSON strings, `EXAMPLES` with the ten input–answer pairs, `N_PROMPTS` with the number of pooled prompts, `TABLE` with the rows. `TABLE_NOTE` is "The table lists all 1,024 heads." in the *all* condition and "The table lists 128 of the 1,024 heads; the others are scored in separate requests." in a batch.

```
You are helping with an interpretability study of a language model with 1,024 attention
↪ heads. We want to find the heads that represent a TASK, that is, what the model has been
↪ asked to do.
```

```
# The task

<<DESCRIPTION>>

The model is asked to do it with instructions like these ({input} stands for the item):
<<TEMPLATES>>

Examples of items and their correct answers:
<<EXAMPLES>>

# What was measured

The model was given <<N_PROMPTS>> prompts of this task, made with the instructions above,
↪  that it answered correctly. At the last token of each prompt, just before the model
↪  writes its answer, we read the output of each head as a list of vocabulary tokens: the
↪  20 tokens that this head's output pushes the model most towards. For each head, the
↪  table at the end gives:
- answer: the share of prompts on which that prompt's own correct answer (its first token)
↪  is among the head's 20 tokens;
- input: the share of prompts on which that prompt's input (its first token) is among the
↪  head's 20 tokens;
- tokens: the 20 tokens that are most often among the head's 20, each with the share of
↪  prompts on which it is there (1.00 = every prompt, .05 = one prompt in twenty).
Tokens are written as JSON strings; a token that begins with a space is shown with the
↪  space. Head ids are anonymous and the heads are in random order: neither says anything
↪  about where a head is in the model.

# How to score a head

Give every head a score from 0 to 100 for how clearly its tokens, across the prompts,
↪  represent the task.
- A head scores high when its tokens name the task, or the relation between the input and
↪  the answer, or the kind of answer without being answers (for example, a word meaning
↪  "city" when the answers are cities). A word that names the task counts even when the
↪  same word is in the instruction.
- The more of its tokens do this, and the higher their shares, the higher the score.
- A head scores 0 when none of its tokens has to do with the task: punctuation, formatting,
↪  numbers, function words, code, pieces of words, or unrelated words.
- Answers and inputs by themselves do not represent the task: a head whose tokens are only
↪  answers of this task, or only inputs of its items, scores low.

Do not lower the score of a head because it also shows answers or inputs. Mark it instead,
↪  in two lists:
- answer: the heads whose tokens include answers of this task (for any of its items);
- input: the heads whose tokens include inputs of its items.
Decide these marks by reading the head's tokens, not from its answer and input shares.

An example, for a task that is not in this study ("give the chemical symbol of an element"):
- " symbol", " element", " chemical", " abbreviation": high score, in neither list;
- " symbol", " element", " Fe", " Au": high score, in the answer list;
- only " Fe", " Au", " Na": low score, in the answer list;
- only " iron", " gold", " sodium": low score, in the input list.

# What to do

<<TABLE_NOTE>> Look at each head in turn, and score every one of them, including the heads
↪  that score 0.

# Answer format

Reply with JSON only, with no other text, in this form:
{"scores": {"<head id>": <score>, ...},
 "answer": ["<head id>", ...],
```

```
 "input": ["<head id>", ...],
 "notes": {"<head id>": {"tokens": [<the tokens that support the score>], "why": "<one short
 ↪  sentence>"}, ...}}
"scores" has one entry for every head in the table, in the order of the table, including the
↪  heads that score 0; a score is a whole number from 0 to 100. "answer" and "input" list
↪  the heads with that mark. "notes" has an entry for every head with a score of 50 or more.

# Heads

<<TABLE>>
```

B Example rows of the table

[TBD: rows of the English–German table, half A, both readouts, once the top-20 readings exist]

Decision log

2026-11-06 The LLM sees readings of the choosing half of E4's split; the checks run on the other half, in two folds, so that the sets sit beside E4's on the same items.

2026-11-06 Each head's row gives the answer share and the input share beside its tokens, since answers change from prompt to prompt and an answer head's most frequent tokens can look like noise.

2026-11-06 Anonymous head ids in random order, so that the choice rests on the tokens, not on what the LLM already believes about the layers of task heads.

2026-11-06 Two conditions: all heads in one request, and batches of 128 heads.

2026-11-06 Every output of the LLM is recorded, including replies that fail the check.

2026-11-06 A pilot on one task before the 14. In the pilot, each request of the *all* condition is repeated once to measure stability.

2026-11-06 Words that name the task count even when they are in the instruction; words for the kind of answer are in scope.

2026-11-06 The LLM scores every head instead of choosing heads; the sets are the highest-scored heads. This replaces a first draft in which the LLM ranked 40 heads (and, in batches, labelled heads and nominated up to 20 for a final ranking), and left out answer and input heads itself.

2026-11-06 Answer heads are removed after the requests, by script, as the heads with an answer share of 0.10 or more; heads that show the input are not removed.

2026-11-06 Reasoning effort medium (first draft: high).

2026-11-06 Pilot, first run: the prompt asked for one JSON object per head. The batch requests came back complete, but in the requests over all heads GPT-5.5 listed only the heads it judged relevant (1 to 67 of 1,024 in 7 replies) and left out the rest; its reasoning summary gives the length of the reply as the reason (about 25,000 tokens). Every one of these requests came back complete on a retry with the error appended (about 23,000 output tokens). Version 1 cost $17.72 for 47 replies. In one batch its summary shows the *answer* and *input* marks set from the shares, not from the tokens. Prompt version 2: scores as one object {id: score} (about 5 tokens per head), the marks as two lists decided

from the tokens, notes only for scores of 50 or more; all 40 pilot requests are sent again with it, so that both conditions have the same prompt without an error appended. Its first request over all heads came back complete at once (9,500 output tokens). The replies to version 1 stay in the log (`prompt_v1.txt`).

2026-11-06 Pilot task English–German, the task on which no set of 20 or 40 heads of E4 has an effect in the first check; country–capital next if it shows none.

2026-11-07 Pilot: larger sets (60, 80 and N_t heads) from the same scores, after the first patching showed no effect of any set of 20 to 40 heads in the first check; no new requests. Built only where every head has a score above 0 in both folds: J-head in both conditions and Logit-head in batches (on English–German, 131 to 180 and 120 to 122 heads above 0), not Logit-head with all heads at once (40 to 64).

References

Guy Davidson, Todd M. Gureckis, Brenden M. Lake, and Adina Williams. Do different prompting methods yield a common task representation in language models? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.12075.

Zhuonan Yang, Jacob Xiaochen Li, Francisco Piedrahita Velez, Eric Todd, David Bau, Michael L. Littman, Stephen H. Bach, and Ellie Pavlick. Shared lexical task representations explain behavioral variability in LLMs. In *International Conference on Machine Learning (ICML)*, volume 306 of *PMLR*, 2026. arXiv:2604.22027.